

Enhancing Coverage and Localization in Multi-Altitude UAV Active SLAM Using Recurrent Proximal Policy Optimization

Anonymous Author(s)

Abstract

We present a reinforcement learning framework that teaches an Unmanned Aerial Vehicle (UAV) to explore an unknown three-dimensional space while keeping its own position estimate trustworthy—deciding, at every moment, not only where to fly horizontally but at which altitude. A single recurrent policy learns to trade off map coverage against localization uncertainty, reaching $91.90 \pm 1.72\%$ coverage with balanced exploration of every altitude layer, and runs in 1.20 ms per decision on a single UAV-class CPU core.

For a UAV to map an unknown environment on its own—a warehouse, a tunnel, a disaster site—it must do two things at once that pull against each other. It must keep moving into unseen space to grow the map, and it must move in a way that lets its onboard Simultaneous Localization and Mapping (SLAM) system stay confident about where it is, because a map built on a drifting pose estimate is worthless. This coupling of *coverage* and *localization* is the heart of active SLAM. In three dimensions the problem is harder still, because the vehicle must also choose its altitude: the same floor looks very different, and is worth exploring to very different degrees, when viewed from one metre or four, and shelves or structure visible from one height may be invisible from another. Classical planners sidestep this by collapsing the world onto a single horizontal slice and scheduling altitude by hand, and by scoring candidate moves with hand-tuned cost functions. Such systems are transparent but brittle: their weights must be re-tuned for every new environment, and they cannot learn, from experience, how to balance pushing into new territory, seeking the loop closures that correct drift, and spreading effort evenly across heights. The result, in practice, is that a hand-tuned controller tends to over-explore whichever layer is easiest and quietly abandon the rest.

This work asks whether a single neural policy can learn all of these concerns together. The central idea is to treat altitude as a decision in its own right. We decompose three-dimensional exploration into a horizontal motion command and a separate “altitude vote”; the vote only changes layers when it is confident, and a minimum-dwell rule prevents the policy from flickering between heights faster than the mapping back-end can absorb. The current altitude is fed back into what the policy sees, so one recurrent controller

arbitrates both kinds of movement coherently rather than handing altitude off to a fixed schedule.

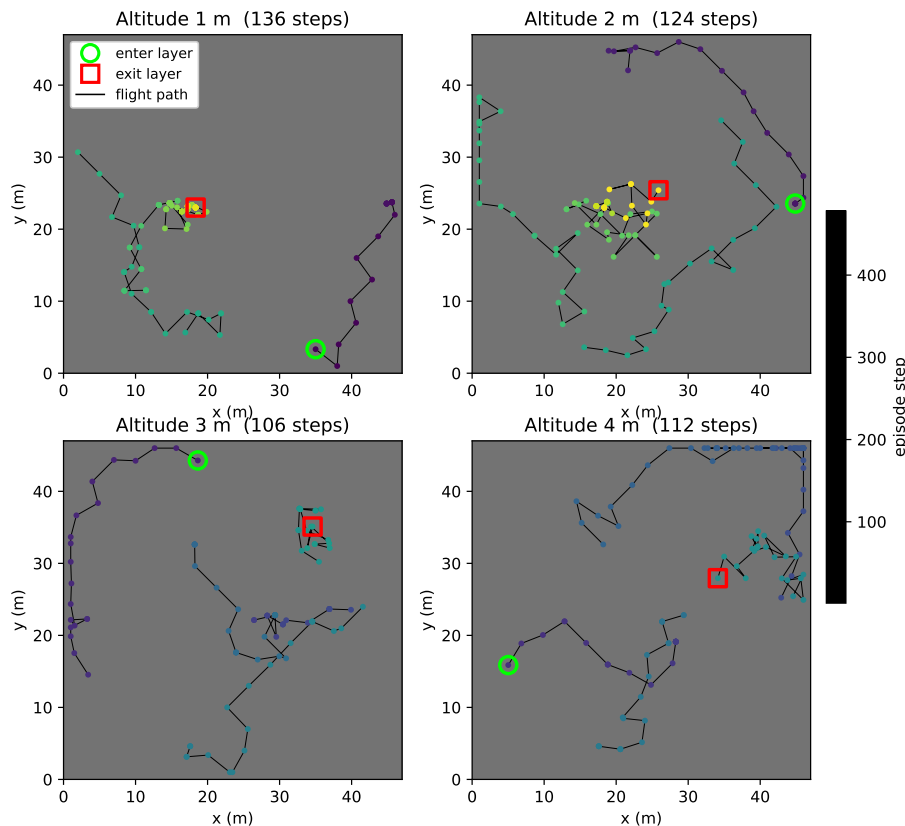
What the policy sees is deliberately shaped for this task. A position-aware convolutional encoder reads a small multi-channel map—current occupancy, where the vehicle has already been, a fading trail of its recent path, and summaries of the least-explored layers—together with a handful of scalar signals such as the current localization uncertainty and the direction of the nearest unexplored frontier. Crucially, the map features and these scalar signals are fused *before* the policy commits to a move, so it can reason that it should, say, steer toward a particular frontier *because* its uncertainty is rising. A recurrent memory then carries intent across the whole flight, letting the agent commit to a multi-step plan—“sweep this layer thoroughly, then climb”—that a memoryless controller could never express.

The behaviour we actually want is encouraged through the reward the policy is trained on. Beyond simply rewarding newly seen space, we add a term that responds to the step-by-step change in localization uncertainty. Loop closures—the moments when the vehicle re-recognizes a place and its SLAM estimate snaps back into agreement—are rare, so a reward that fires only on those moments is too sparse to learn from; by instead shaping the continuous uncertainty signal, we turn that rare event into a dense, learnable pressure that nudges the agent toward trajectories that keep its pose estimate sharp. A second ingredient, a per-altitude “breadth” incentive, rewards the policy the first time each layer becomes well explored, so that no height is neglected in favour of the easy middle layers.

Trained and evaluated on fifty held-out maps it had never seen, across three independent runs, the learned policy reaches $91.90 \pm 1.72\%$ coverage. This is a large, statistically significant improvement over six classical exploration strategies and over an otherwise identical reinforcement learning agent trained without our reward design. The gains are not only in the headline number: the policy exceeds 85% coverage on *every* altitude layer, where the strongest classical planner collapses to about 70% on the top layer, and it reaches the half-way coverage mark roughly twice as fast in flight time, all while holding localization uncertainty as low as the best baseline. Controlled ablations make the roles of the two key ingredients clear: removing the breadth incentive does not change average coverage much but makes per-altitude effort wildly inconsistent from run to run, while removing the loop-closure-related reward leaves coverage intact but lets localization uncertainty climb—confirming that each ingredient does the job it was designed for. Figure 1 shows a representative flight: the agent distributes its time almost evenly across all four altitude layers and revisits mapped regions just often enough to keep itself well localized.

It is worth being precise about what these results mean, because in learning-based exploration it is easy to report numbers that do not generalize. Every

figure quoted above is measured on maps the policy never saw during training, and averaged over three independently trained agents, so the gains reflect a learned strategy rather than memorization of particular layouts or a single lucky run. The comparison is also fair in the strongest sense: the learning baseline we improve upon shares our exact network, training budget, and curriculum, and differs only in the reward design, so the improvement isolates the contribution of the ideas above rather than of more compute or a bigger model. The advantage over the classical planners holds by a wide margin across every map, not merely on average.



A representative learned flight on an unseen map (one grid cell is 1 m). Each panel is one altitude layer, showing obstacles (grey) and the vehicle's path while flying at that height, coloured from the start of the episode (dark) to the end (bright); circles and squares mark where the vehicle enters and leaves each layer. The policy spreads its effort evenly across all four layers rather than collapsing onto a single convenient slice.

Because the eventual goal is a real aircraft, the policy is deliberately small and fast: about two million parameters, evaluating in just over a millisecond on a single ordinary CPU core with no graphics accelerator, which leaves ample headroom on the kind of modest computer a UAV can carry. The same controller already runs, without any retraining, inside a high-fidelity simulator coupled to a real visual-SLAM back-end, where it explores with stable

tracking. The principal obstacle to flying on real hardware is motion smoothness—real visual SLAM dislikes the abrupt reversals an idealized grid agent can make freely—and we outline a concrete path to closing that gap, from smoothing the commanded motion through to on-board flight tests.

The significance of this work extends beyond the warehouse setting we study. The same coupling of coverage, localization, and altitude arises wherever a UAV must map a structured three-dimensional space on its own—inspecting the underside of a bridge or the inside of a storage tank, searching the floors of a partially collapsed building, or surveying vegetation at several canopy heights. In all of these the easy failure mode is the same: a planner that pours effort into whichever layer is convenient and neglects the rest, or that flies in a way which lets its position estimate drift until the map no longer fits together. By making altitude a first-class decision and rewarding the agent for actively protecting its localization, the policy learns the balanced, self-correcting behaviour these missions demand.

In short, by letting a single recurrent policy own the altitude decision and by shaping its reward around the localization signal that classical methods only ever react to, we obtain exploration that is more complete, better balanced across heights, faster, and light enough to run on the vehicle itself—a practical step toward UAVs that map unknown three-dimensional spaces reliably on their own.